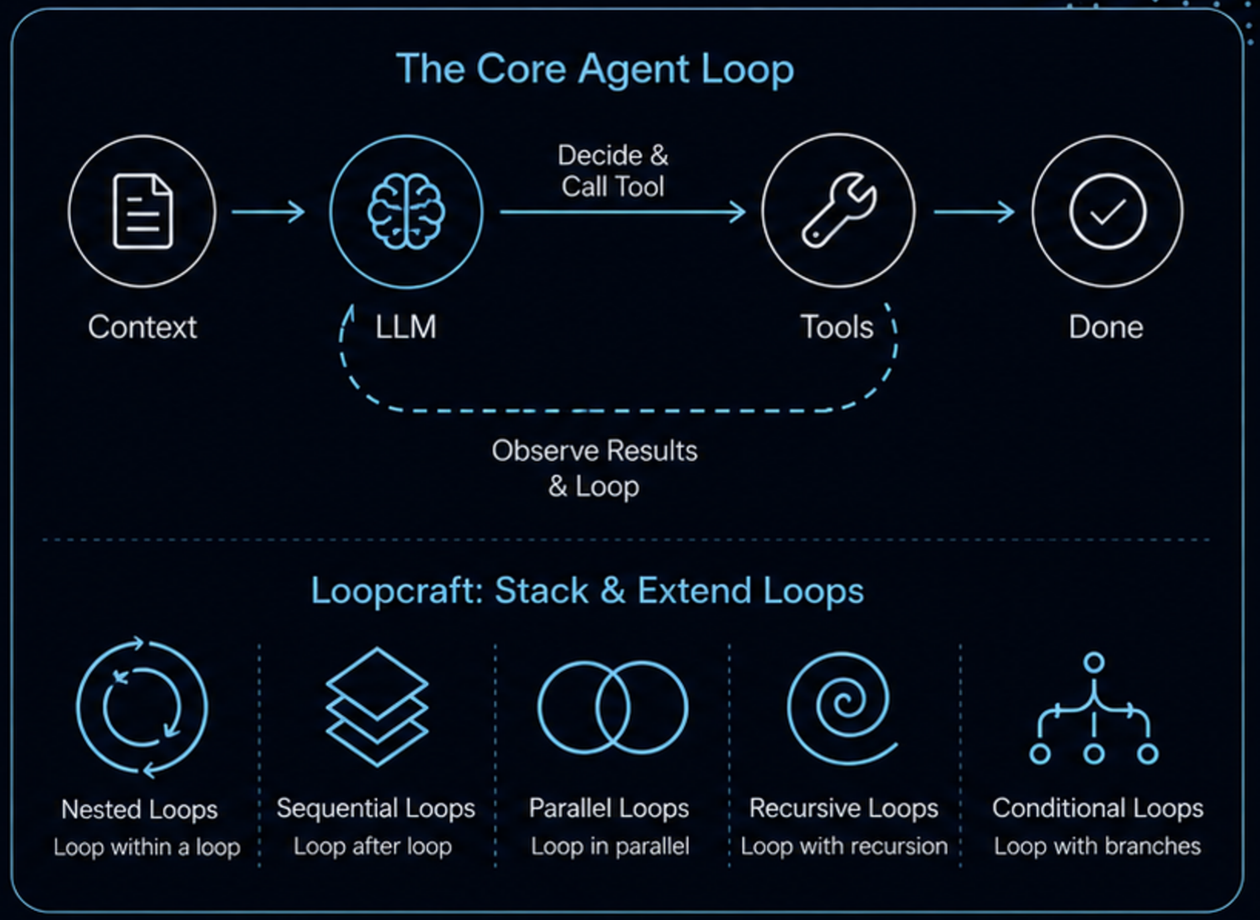
The Art of Loop Engineering

LangChain

The Art of Loop Engineering

Agents are useful because they help us automate work by taking actions in the real world. But getting agents to do valuable work reliably takes more than just a good model: it requires a carefully designed harness that's fit to a set of tasks.

The core agent algorithm is simple: give the LLM context and let it call tools in a loop until it's done. This is the most fundamental loop. But it's far from the only loop that powers agents. @swyx recently wrote a great piece on "loopcraft: the art of stacking loops", the idea that you can stack and extend loops to build more effective agents.



LangChain

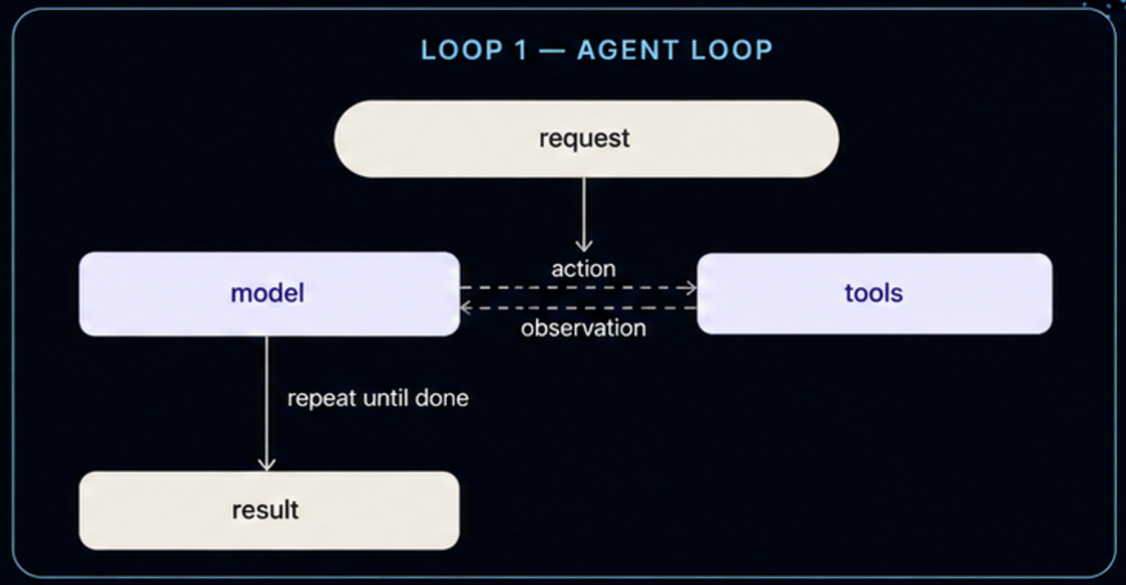
The Art of Loop Engineering

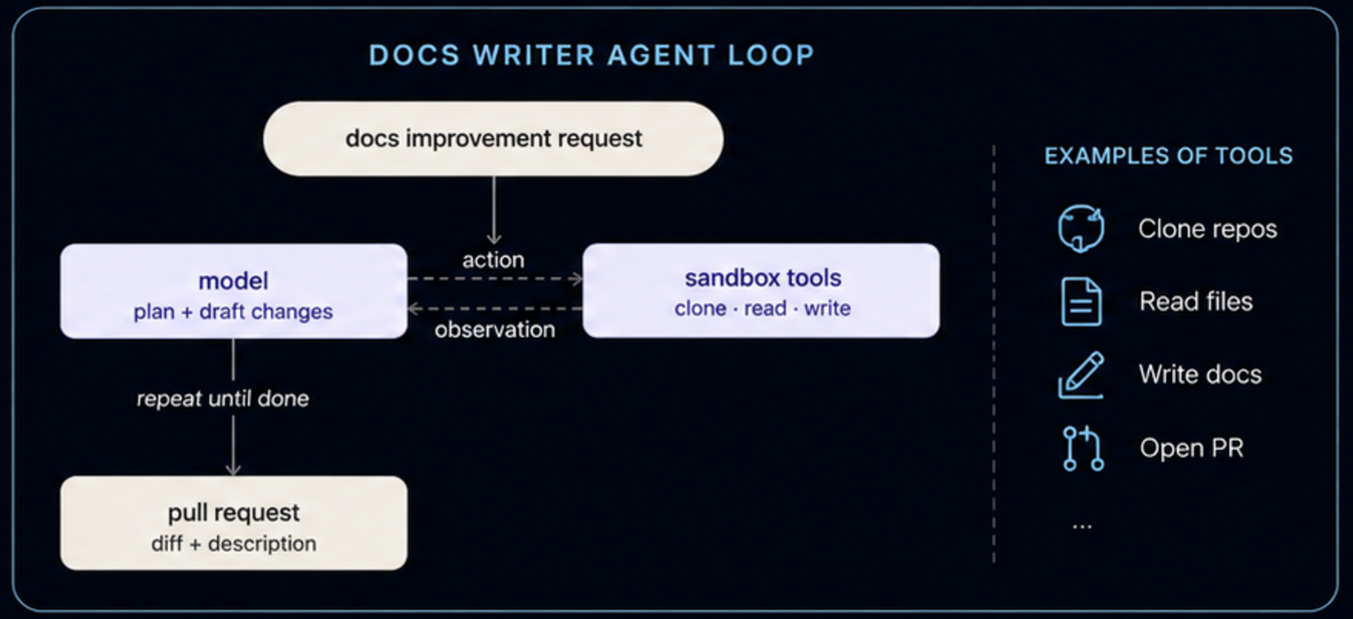
Loop 1: The Agent

At its core, an agent is just a model calling tools in a loop until a task is complete.

This is what LangChain's **create_agent** gives you. Pick any model, plug in tools, and you have a working agent loop. Tools are what give the agent the power to take action in the real world.

Take our internal docs agent as an example (which we'll use as a motivating example for the rest of this blog). At the first loop level, it receives a request for a documentation improvement, the model plans and draft changes, and it uses tools to clone repos, read files, write docs, open a pull request, etc.





The Art of Loop Engineering

Level 2: Verification loop

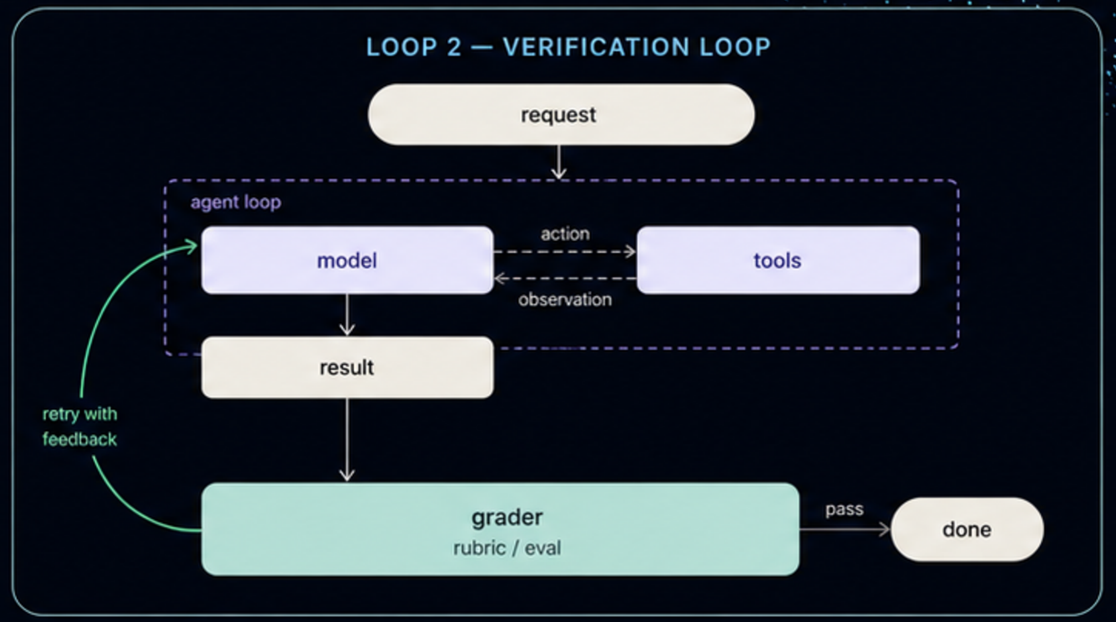
The agent loop gets work done, but it doesn't always produce correct or consistent work on the first pass. When consistency matters, it's often useful to wrap it in a verification loop that checks the output and sends feedback back to the model when it falls short.

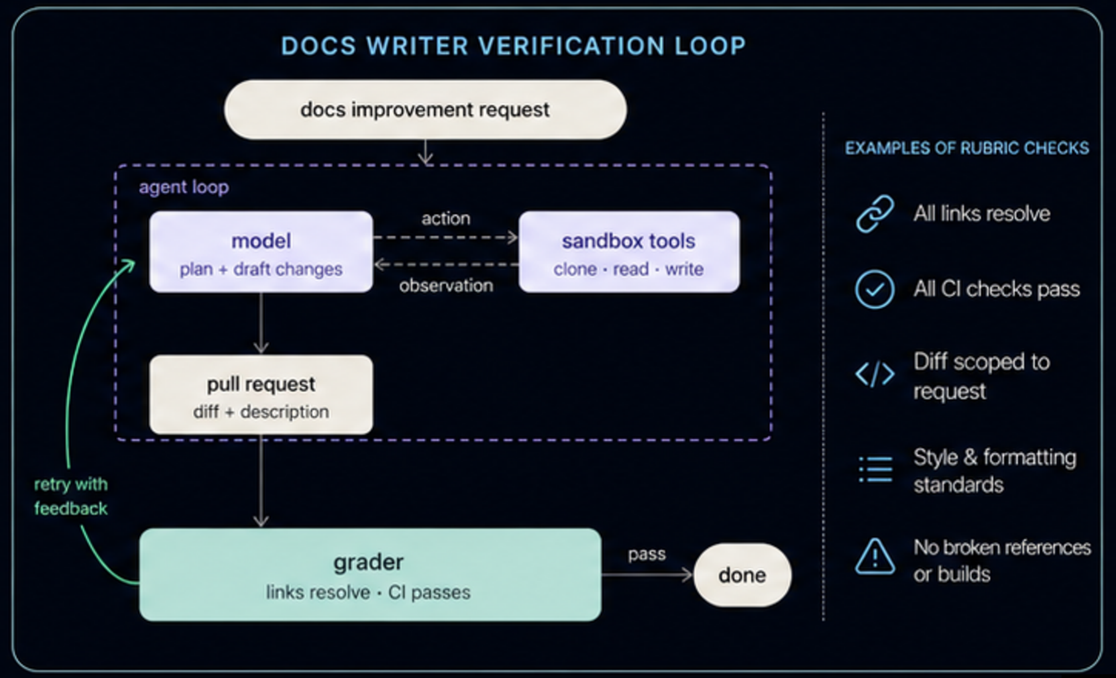
The verification loop adds a grader: something that checks the agent's output against a rubric and, if it fails, sends the result back with feedback. Graders can either be deterministic or agentic (LLM as a judge is a classic example, here).

RubricMiddleware handles this pattern, or you can wire it up with an after_agent hook on create_agent.

For our docs writer example, the grader runs tests after each attempt, checking that all links resolve, all CI checks pass, and the diff is scoped to what was actually requested. No manual review needed to catch those classes of error.

One tradeoff: adding verification increases latency and cost per run. It's worth it when quality matters more than speed, which is most production use cases.





LangChain

The Art of Loop Engineering

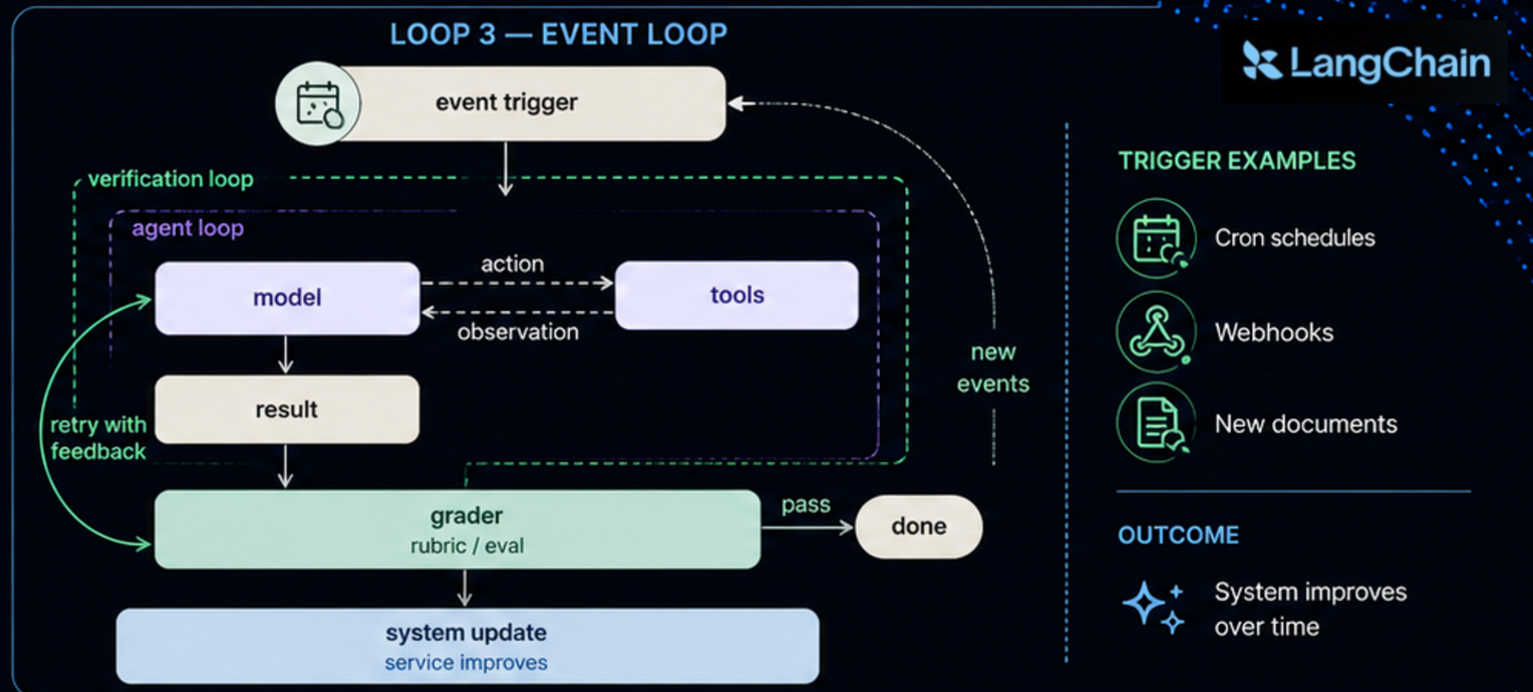
Level 3: Event driven loop

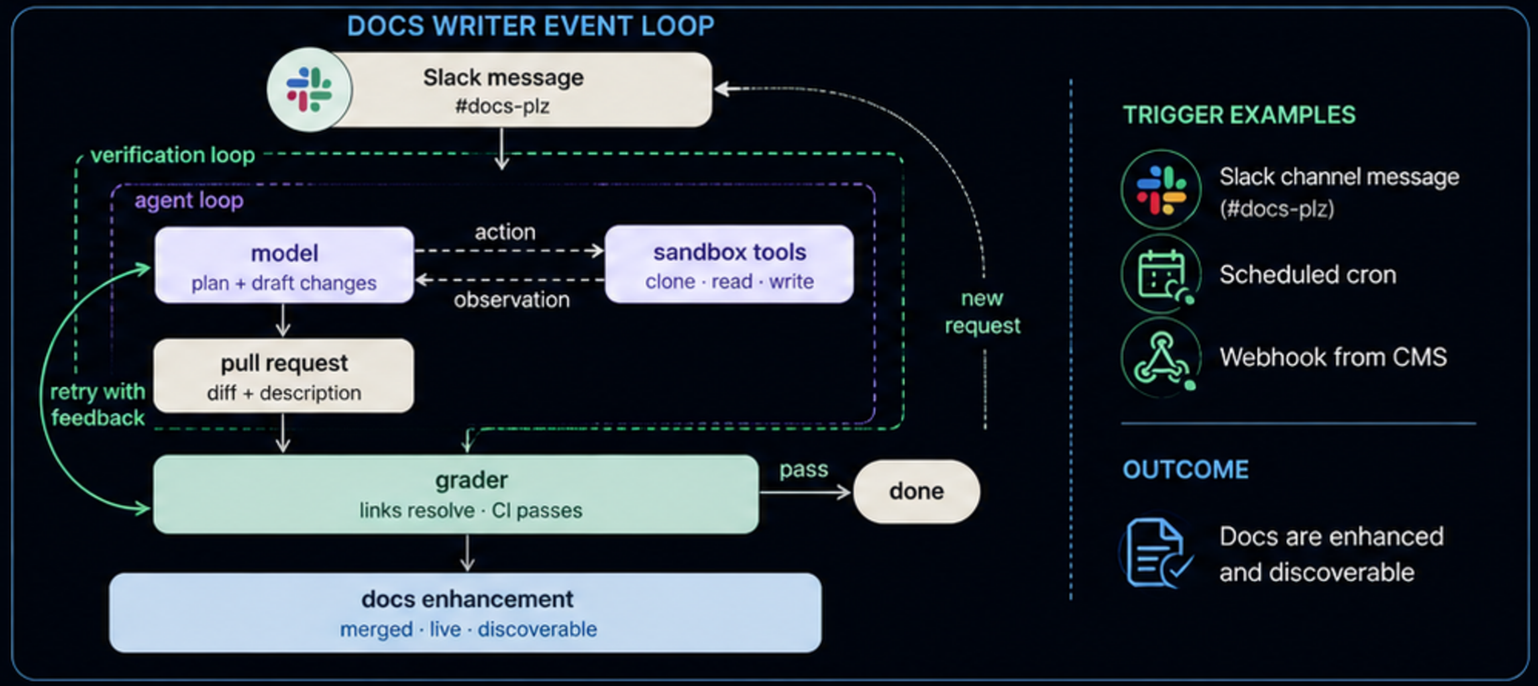
One of the most important parts of agent development is the integrations layer: connecting your agent to your ecosystem so that it can run in the background.

The event-driven loop connects your agent to your ecosystem. An event fires — a new document lands, a schedule triggers, a webhook arrives — and the agent runs. The agent isn't something you invoke manually; it's a component running continuously inside a larger system.

LangSmith Deployment supports the trigger infrastructure, including support for cron schedules and webhooks. One popular example of crons in action is "heartbeats" in openclaw, which turn your agent into an always-on, proactive assistant.

Our docs agent is powered by Fleet, our no-code agent builder. Fleet's channels and schedules handle event-driven and cron-style triggers. We use a channel to fire off the docs agent whenever a message is sent in our #docs-plz Slack channel.





The Art of Loop Engineering

Level 4: Hill climbing loop

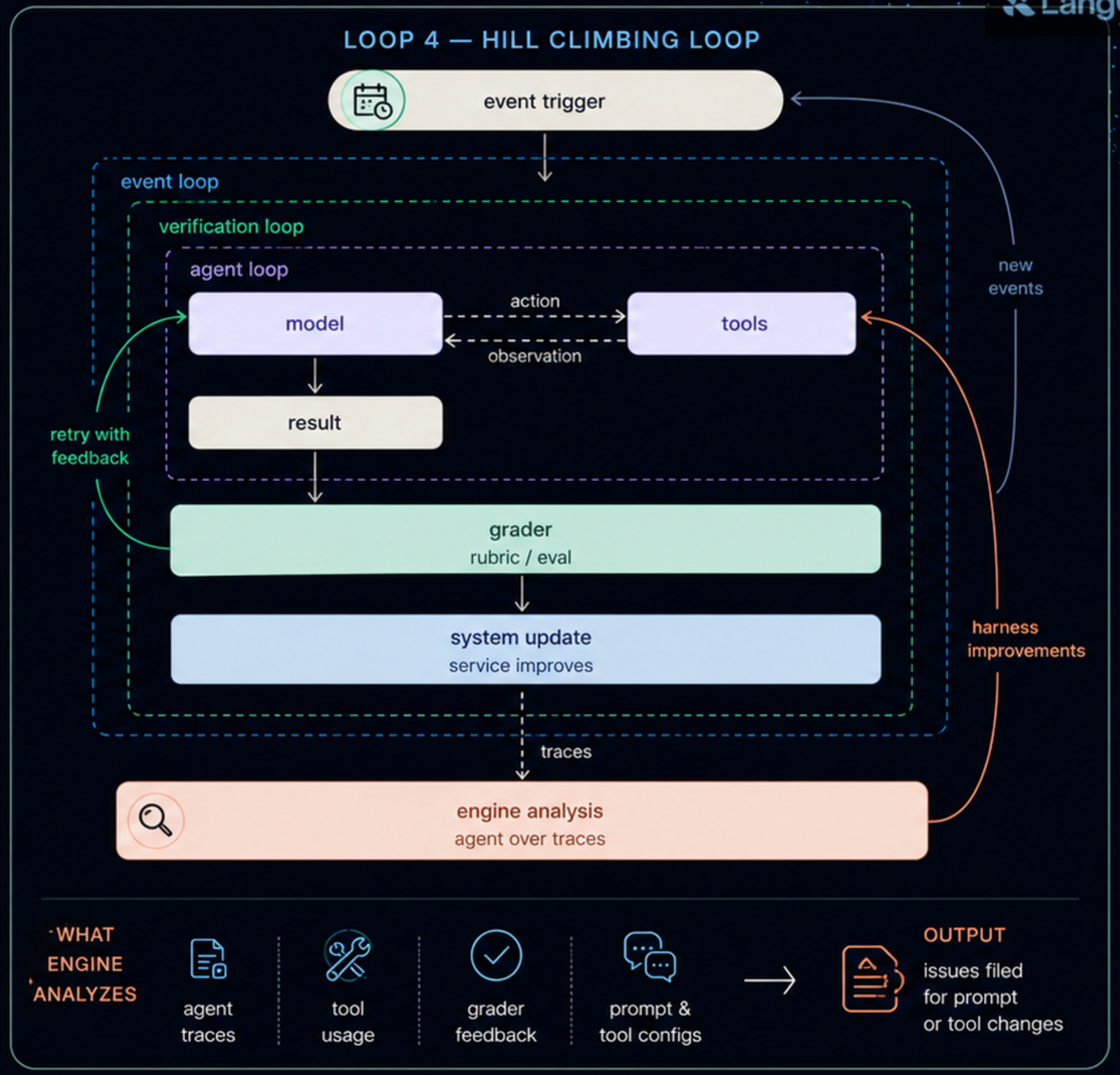
The first three loops automate work. The fourth (and arguably most important) automates improvement!

Every agent run produce a trace: a record of what the model did, the tools it called, grader feedback, etc. Those traces contain high value signal regarding what's working and what isn't.

The hill climbing loop runs an analysis agent over those traces and uses the findings to rewrite the harness with improved configuration. That can include prompt/tool tweaks or grader tweaks.

In LangSmith, you can use Engine, our trace analysis agent, to instrument this fourth loop.

Wrapping up the docs agent analogy, we run Engine over the docs agent traces to detect any issues. When multiple traces signal a potential problem, an issue is filed requesting changes to the offending prompt or tool.



The Art of Loop Engineering

The key move

The key move here is that the return arrow doesn't just loop back to the top — it reaches inside and updates the agent loop directly.

Each cycle of the outer loop makes the inner loops more effective.

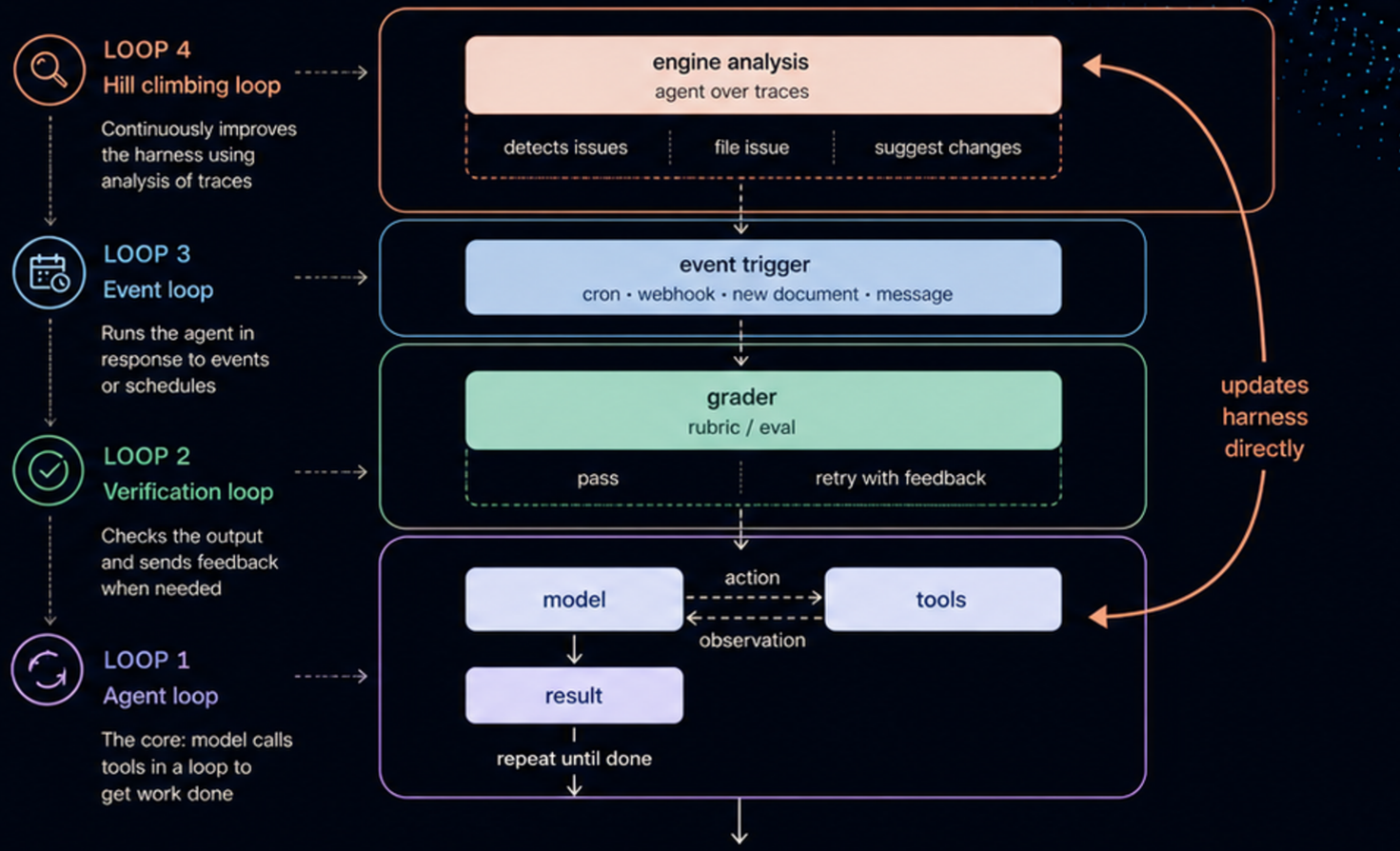
Looking forward

Prompt and tool configuration are the most simple things to improve, but they're not the only options. For teams running open-weight models, the hill climbing loop can feed into RL fine-tuning, using trace or eval outcomes as training signal to improve the model itself. Auxiliary context like memory and retrieved skills can be improved the same way.

The loop is the pattern; what it optimizes is up to you.

Human oversight and expertise

Automation doesn't mean removing humans from the loop. At every level, there are natural points where human oversight adds value. An automated grader can check whether links resolve; it takes a human to notice the framing is wrong for the audience. That kind of judgment, earned from context, experience, and taste, is exactly where human review earns its place.



HUMAN TOUCH POINTS

Agent loop
Require human input before sensitive actions or tool calls

Verification loop
A human can act as the grader for sensitive workflows

Application loop
A human can approve outputs before they're returned to the end user

Hill climbing loop
Harness improvements can flow through human review before deployment

All of LangChain's open source frameworks make adding a **"human in the loop" a first class primitive.**

LangChain

The Art of Loop Engineering

Putting it all together

In case you'd prefer a more tabular view, here's how those four loops stack together:

This is what loop engineering — or loopcraft, as @swyx puts it — actually looks like in practice. AI leaders like Steipete, Boris, and Andrej have all arrived at the same conclusion: **the potential in agents is in the loops you build around them.**

We've been thinking about loops 1 and 2 for a while. But focus should pivot to loops 3 and 4 where **value compounds** by embedding agents into your ecosystem that continuously improve in response to your criteria.

LOOP	WHAT IT DOES	IMPACT	LANGCHAIN PRIMITIVE
1: Agent loop (model + tools)	Model calls tools repeatedly until a task is complete	Automate work	create_agent, any LangChain-supported model
2: Verification loop (agent + grader)	Agent runs, output is scored against a rubric, retried with feedback if it fails	Ensure quality	RubricMiddleware
3: Event loop (verification + system)	Events trigger agent runs that update a real system	Work at scale	LangSmith Deployment / Fleet channels
4: Hill climbing loop (system + engine)	Production traces feed an analysis agent that improves the harness config	Continuous improvement	LangSmith Engine

LangChain